Lec102

Dynamic Programming

Those who forget the past, are condemned to repeat it.

Methods

1) Top down → Recursion + Memoizatn
2) Buttom-up → Tabulation.
   + space optimisation.

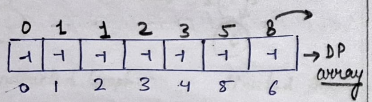
Fibonacci Series

0 1 1 2 3 5 8 13 21 ....

$f(n) = f(n-1) + f(n-2)$

DP is used when : you st see overlapping of subproblems.

since, in fibo. series → only 1 parater is passed, 1D array can be used to DP.



dp array → [n+1] size.

(II) dp[n] = f(n-1) + f(n-2)
return dp[n]

(III) dp[n+1];

```
int fib (int n, vector <int> &dp) {
  // base case
  if (n <=1) return n;
    return n;

  if (dp[n] != -1)
      return dp[n];

  dp[n] = fib(n-1,dp) + fib(n-2, dp);
  return dp[n];
}

int main () {
    int n;
    cin >> n;
    vector <int> dp(n+1);
    for (int i=0; i<=n; i++)
        dp[i] = -1;

    cout << fib(n, dp) << endl;
    return 0;
}
```

T.C → O(n)

S.C → O(N) + O(N) = O(N)

*****

NOTE : Think step by step;

Recursion + memorisation → Tabulation → Space Optimisation.

Tabulation method.

```
fib(n) {
```

(I) dp[n+1]; all values = -1

(II) dp[1] = 1;
dp[0] = 0; } initial values.

(III) for (i=2; i<=n; i++) {
    dp[i] = dp[i-1] + dp[i-2];
}

return dp[n];
}

T.C = O(n)

S.C = O(n)

code

```
int main(){
    int n;
    cin >> n;

    vector <int> dp(n+1);
    dp[0] = 0;
    dp[1] = 1;

    for (int i=2; i<=n; i++)
        dp[i] = dp[i-1] + dp[i-2];

    cout << dp[n] << endl;
    return 0;
}
```

Optimisation → use variables Prev1 & prev2

update curr = Prev1 + Prev2

shift Prev2 = Prev1
Prev1 = curr.
repeat.

TC = O(n)
SC = O(1)

Lec103 Only Tabulation

Q.) Climbing staire Soln

```
int climbingStair (int n) {
    if (n==1) return 1;
    vector <int> dp(n+1);
    dp[2] = 2;
    dp[1] = 1;
    for (int i=3; i<=n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

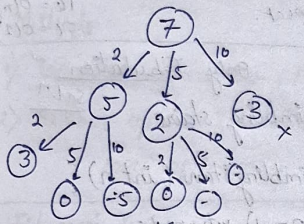
Lec104 Q.) Minimum Coins.

i/p = coins [1, 2, 5] → Infinite supply.
amount = 11

output : 3
↳ 5, 5, 1 = 11

Eg. Target = 7
coins = [2, 3, 5] / [3, 5, 10]

Approach



recursion

// Base case
if (target == 0)
    return 0;
if (target < 0)
    return INT_MAX;

int mini = INT_MAX;

// recursive called for all coins.
for (int i=0; i<n; i++){
    int ans = funct^n (amt - coin[i]);
    if (ans != INT_MAX)
        mini = min (mini, ans+1)
}
return mini.

code

x → target

int solveTabulat^n (vector<int> &coins, int x) {

vector<int> dp (x+1, INT_MAX);
dp[0] = 0;

for (int i=1; i<=x; i++) {
// store ans for each reachable target
for (int j=0; j < coins.size(); j++){
if (i - coins[j] >= 0 && dp[i-coins[j]] != INT_MAX) {
dp[i] = min (dp[i], 1 + dp[i-coins[j]]);
}
}

if (dp[x] == INT_MAX)
return -1;
return dp[x];
}

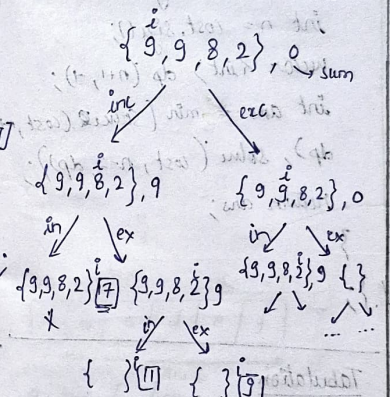
int minElements (vector &nums, int x) {
ret solveTabulat^n (&nums, x);
}

lec 105

Q.) Maximum Sum of Non Adjacent Elements.

eg arr = {9, 9, 8, 2}

approach, only two choice → Include / exclude



code

int solve (vector &nums, int n, vector &dp) {
if (n < 0) return 0;
if (n == 0) return nums[0];
if (dp[n] != -1) return dp[n];

int incl = solve (nums, n-2, dp) + nums[n];
int excl = solve (nums, n-1, dp);
dp[n] = max (incl, excl);
return dp;
}

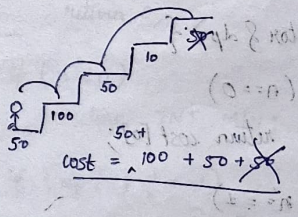
int rob (vector nums) {
int n = nums.size();
vector<int> dp (n, -1);
return solve (nums, n-1, dp);
}
}

dec 103 –

Q.) Minimum cost climbing stairs.

① We can start from stair 0th or 1st

② we have to pay cost of that particular stair to move from that stair.



③ Return min cost to reach top floor / stair.

approach

$f(n) = (f(n-1) + f(n-2))$ min value

$f(n-1)$: cost to reach $n-1$ stairs

$f(n-2)$: cost to reach $n-2$ stairs

also then add cost of $f(n-1)$ or $f(n-2)$ which whoever is minimum (to move to $f(n)^{th}$ stair).

updation will become

$f(k) = \min(f(k-1), f(k-2)) + cost[k]$

code

```
int solve (vector &cost, int n,
vector &dp) {
if (n==0)
    return cost[0];
if (n==1)
    return cost[1];
if (dp[n] != -1)
    return dp[n];
dp[n] = cost[n] + min(solve(
    cost, n-1, dp), solve(cost, n-2,dp));
return dp[n];
}

int minCostClimbingStairs (vector &cost)
{
    int n = cost.size();
    vector<int> dp(n+1, -1);
    int ans = min(solve(cost,n-1,
    dp), solve(cost, n-2, dp));
    return ans;
}
};
```

Tabulation

```
int solve2 (vector &cost, int n){
    vector<int> dp(n+1);
    dp[0] = cost[0];
    dp[1] = cost[1];
    for(int i=2; i<n; i++){
        dp[i] = cost[i] + min(dp[i-1],
                dp[i-2]);
    }
    return min(dp[n-1], dp[n-2]);
}
```

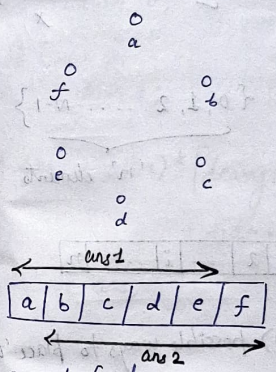
lec 106

Robber House

↳ no two adjacent houses should be broken (robbed).

↳ maximise profit of robber.

↳ Houses are circular.



a and f houses are adjacent.

return max (ans1, ans2)

use code of max^m sum of non-adj elements to calculate ans1 & ans2.

Note

R+mem

↳ solve(n, &dp) {

}

main(){ dp(n+1, -1);

}

Tabulation

```
solve(n) {
    dp(n+1,-1)
    dp[0] = ...   } base cases
    dp[1] = ...
    for(2...n) {
    ____________
    }
    return dp[n];
}
```

lec 107

Q.) Cut Rod in X, Y, Z segment.

Recur + mem

```
SolveMem(int n, x, y, z,
        vector &dp) {
if (n==0)
    return 0;
if (n<0)
    return INT_MIN;
if (dp[n] != -1)
    return dp[n];
int a = solve(n-x, x, y, z, dp);
int b = solve(n-y, x, y, z, dp);
int c = solve(n-z, x, y, z, dp);
```

```
dp[n] = max(a, max(b, c));
return dp[n];
}
```

Tabulation

```
solveTab (int n, x, y, z) {
vector <int> dp(n+1, INT_MIN);
dp[n] = 0;
for (int i=1; i<n; i++){
    if ( dp[i] i-x >=0)
        dp[i] = max(dp[i], dp[i-x]+1);

if (i-y >=0)
    dp[i] = max(dp[i], dp[i-y]+1);

if (i-z >=0)
    dp[i] = max(dp[i], dp[i-z]+1);

}
if (dp[n] < 0)
    return 0;
else {
    return dp[n];
}
}
```

---

lec 108

Q.) Count Derarrangements.

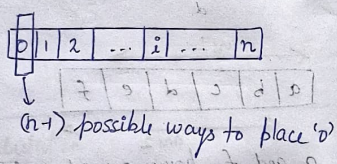
(no element appears in its original place)

e.g. {0, 1, 2, 3}

{3, 2, 1, 0} ✓ {3, 1, 2, 0} ✗

i/p → {0, 1, 2 .... N-1}

'n' elements.



ans = (n-1) * [Solution of subproblem]

lets place 0 at ith index.

0, i swapped

0 & ith element not swapped.

recursion relation

$$F(n) = (n-1) * (F(n-1) + F(n-2))$$

---

note // base cases

F(1) = 0 & F(2) = 1

code - recursion

```
int dearrg (int n) {
    if (n==1)
        return 0;
    if (n==2)
        return 1;
int ans = (n-1) * [dearrg(n-1)
                   + dearrg(n-2)];
return ans;
}
```

---

rec + mem

Tabulation

```
int Tabulation (int n){
    vector <int> dp(n+1, -1);
    dp[1] = 0;
    dp[2] = 1;
    for (i=3; i<=n; i++) {
    dp[i] = (i-1)* [dp[i-1] +
                    dp[i-2]];
    }
    return dp[n];
}
```

---

lec 109

Q.) Painting fence Algorithm

Solve(n) = solve(n-2) * (n-2)

recursion

```
int solve (int n, int k) {
    if (n==1) {
        return k;
    if (n==2) {
        return k*(k-1) + k;
int ans = (k-1)* solve(n-2,k) +
          (k-1)* solve(n-1,k)
return ans;
}
```

(k^2)

recc + memoi

```
int soluMem (n, k, dp) {
    // base cases

    if (dp[n] != -1)
        return dp[n];

    dp[n] = add(mul ..... )
    return dp[n];
}
```

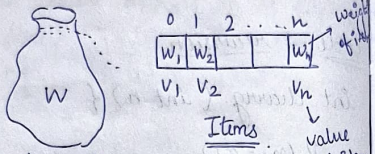
```
int soluTab (int n, int k) {
    vector<int> dp(n+1, 0);
    dp[1] = k;
    dp[2] = add(k, mul(k, k-1));

    for (i=3; i<=n; i++) {
        dp[i] = (dp[i-2]*(k-1)) +
                (dp[i-1]*(k-1))
    }
    return dp[n];
}
```

---

lec 110 2-D D.p

Q.) Knapsack problem



maximise ΣV_i
while $w_1 + w_2 \ldots = W$

Question

---

eg $W = 5$

$w_i \rightarrow$	1	2	4	5
$v_i \rightarrow$	(5)	(4)	(8)	(6)

ans = {1, 4} = W = 5

Σv_i = (13)

---

recursion + memoi

```
int solveR(vector weight, vector
value, int index, int capacity,
vector<vector>&dp) {

    if (index == 0) {
        if (weight[0] <= capacity) {
            return value[0];
        }
        else {
            return 0;
        }
    }

    if (dp[index][capacity] != -1)
        return dp[i][cap];

    int include = 0;
    if (weight[index] <= capacity)
        include = value[index] +
        soluM (weight, value, index-1,
        capacity, dp);

    int exclude = 0 + soluM (weight,
    value, index-1, capacity, dp);

    dp[index][capacity] = max (
                include, exclude);
    return dp[index][capacity];
}

int knapsack (weight, value, n, W) {

    vector<vector> dp (n, W+1, -1)
    return soluM (weight, value, n-1,
                W, dp);
}
```

Tabulation

```
int soluTab (weight, value, n,
            capacity) {

    vector<vector> dp (n, cap+1, 0));

    // base condition
    for (int w = weight[0]; w <= cap;
                w++) {
        if (weight[0] <= capacity) {
            dp[0][w] = value[0];
        } else {
            dp[0][w] = 0;
        }

    for (int index = 1; index < n; i++) {
        for (int w = 0; w <= capacity;
                w++) {

            int include = 0;
            if (weight[index] <= w) {
                include = value[index] +
                dp[index-1][w - weight[index]];
            int exclude = 0 + dp[index-1][w];
            dp[index][w] = max (include,
                        exclude);
            }
        }
        return dp[n-1][capacity];
    }
```

lec 111

Q) combination sum IV

N=3 → {1,2,5}

target = 5

print total numbers of ways to reach target sum (5)

e.g. o/p = 9

① → {1 1 1 1 1}

④ → {1 1 1 2}, {1 1 2 1} {1 2 1 1} {2 1 1 1}

① → {5}

③ → {2,2,1} {2,1,2} {1,2,2}

11 → ✗

⑨ ways

code R+mem

```
int solve (nums, target, dp){
  // base case
  if (target == 0) {
      return 1;
  } else if (target < 0) {
      return 0;
  }
  if (dp [target] != -1) {
      return dp [target];
  }
  // recursive call
  int ans = 0;
  for (int i=0; i<nums.size(); i++) {
      ans += solve (nums, target-nums[i], dp);
  }
  dp [target] = ans;
  return dp [target];
}
```

Tabulation

```
int solveTab (nums, target) {
  vector <int> dp (target+1, 0);
  dp [0] = 1;
  for (i = 1; i<=target; i++) {
      for (int j=0; j<nums.size(); j++) {
          dp [i] += dp [i - nums [j]];
      }
  }
  return dp [target];
}
```

lec 112

Q.) Perfect Squares

n = 100

o/p → 1 {i.e 10*10}

n = 5

o/p → 2 {i.e 1*1 + 2*2}

code R+mem

```
int solveMem (int n, vector &dp){
  if (n==0) {
      return 0;
  }
  if (dp [n] != -1) {
      return dp [n];
  }
  int ans = n;
  for (int i=1; i*i<=n; i++){
      int temp = i*i;
      ans = min (ans, 1 + solveMem (n-temp, dp));
  }
  dp [n] = ans
```

Tabulation

```
int solveTab (int n) {
  vector <int> dp (n+1, INT_MAX);
  dp [0] = 0;
  for (int i=1; i<=n; i++) {
      for (int j=1; j*j<=n; j++){
          int temp = j*j;
          dp [i] = min (dp [i],
              1 + dp [i - temp]);
      }
  }
  return dp [n];
}
```

check i - temp is valid idx or not.

lec 113

Q.) Minimum cost of tickets.

recursion approach:

days

e.g. {1, 4, 6, 7, 8, 20}

i

we have 3 choices each time: 1D, 7D, 30D

costs = [2, 7, 15]

Tabulation n = days.size();

```
int solveTab(n, days, costs, index){
    vector<int> dp(n+1, INT_MAX);
    dp[0] = 0;
    dp[n] = 0;
    for(int k=n-1; k>=0; k--){
        int day1 = costs[0] + dp[k+1];
        int i;
        for(i=k; i<n && day[i]<days[k]+7; i++);
        int day7 = costs[1] + dp[i];
        for(i=k; i<n && days[i]<day[k]+30; i++);
        int day30 = cost[2] + dp[i];
        dp[k] = min(1 min(7,30));
    }
    return dp[0];
}
```

---

lec114 optimisation using

```
queue<pair<int,int>> q;
        day  cost.
```

code

```
int solve(n, days, costs){
    int ans = 0;
    queue<pair<int,int>> month;
    queue<pair<int,int>> week;
    for(int day : days){
        // remove expired days
        while(!month.empty() && month.
              front().first + 30 <= day)
            month.pop();
        while(!week.empty() && week.front
              first + 7 <= day)
            week.pop();
        // add cost for current day.
        week.push(make_pair(day, ans+
                  cost[1]));
        month.push(make_pair(day, ans+
                   cost[2]));
        // step3 ans update
        ans = min(ans+cost[0],
            min(week.front().second,
                month.front().second);
    }
    return ans;
```